

Fiche de révision — Ingénierie du logiciel (S7)

Méthodes clés pour traiter n'importe quel exercice de génie logiciel

1. Cycle de vie logiciel

Modèle	Principe	Quand l'utiliser
Cascade (waterfall)	phases séquentielles, chaque phase se termine avant la suivante	besoins stables et bien connus dès le départ
Cycle en V	cascade + phases de vérification miroir (tests associés à chaque phase de conception)	projets critiques, besoin de traçabilité forte
Itératif / incrémental	on livre des versions partielles successives, chacune enrichie	besoins évolutifs, réduction du risque
Agile / Scrum	sprints courts, feedback client continu, backlog priorisé	besoins flous ou changeants, forte collaboration client

2. Recueil et gestion des besoins

- **Besoin fonctionnel** : ce que le système doit faire (ex. « l'utilisateur peut réinitialiser son mot de passe »).
- **Besoin non-fonctionnel** : contrainte de qualité (performance, sécurité, ergonomie, portabilité...).
- **Méthode type d'exercice** : à partir d'un cahier des charges, extraire les acteurs, les cas d'usage, puis classer chaque exigence en fonctionnelle/non-fonctionnelle et vérifier qu'elle est testable et non ambiguë.

3. Diagrammes UML — quand et comment les construire

Diagramme de cas d'utilisation

Identifier les **acteurs** (externes au système) et les **cas d'usage** (buts atteints). Relations : *include* (comportement toujours exécuté), *extend* (comportement optionnel/conditionnel), généralisation entre acteurs ou entre cas d'usage.

Diagramme de classes

- Association (trait simple) : lien structurel, noter les cardinalités aux deux extrémités (0..1, 1, *, 1..*).
- Agrégation (losange vide) : « a un », les parties peuvent exister sans le tout.
- Composition (losange plein) : « possède », les parties n'existent pas sans le tout (destruction en cascade).
- Héritage (flèche triangle vide) : généralisation/spécialisation, respecter le principe de substitution.
- Méthode : repérer les noms → classes candidates, les verbes → méthodes/associations, les adjectifs/valeurs → attributs.

Diagramme de séquence

Représente les échanges de messages entre objets dans le temps (axe vertical = temps). Bien distinguer message synchrone (flèche pleine, attend la réponse) et asynchrone (flèche ouverte). Les blocs alt/opt/loop encadrent les branches conditionnelles ou répétitions.

Diagramme d'état-transition

Modélise le cycle de vie d'un objet : états, événements déclencheurs, gardes [condition], actions. Vérifier qu'il existe un état initial et qu'aucun état (sauf final) n'est un cul-de-sac non voulu.

Diagramme d'activité

Modélise un flux de traitement/processus métier : actions, branchements (losanges), synchronisations (barres).
Utile pour décrire un algorithme métier ou un cas d'utilisation complexe.

4. Principes SOLID (à appliquer dans un exercice de conception)

Principe	Idée	Signe qu'il est violé
S — Single Responsibility	une classe = une seule raison de changer	la classe change pour des raisons sans rapport entre elles
O — Open/Closed	ouvert à l'extension, fermé à la modification	on doit modifier une classe existante pour ajouter un cas
L — Liskov Substitution	une sous-classe doit pouvoir remplacer sa classe mère sans casser le comportement attendu	la sous-classe restreint des préconditions ou lève des exceptions inattendues
I — Interface Segregation	préférer plusieurs interfaces spécifiques à une seule interface générale	une classe implémente des méthodes qu'elle n'utilise pas
D — Dependency Inversion	dépendre d'abstractions, pas d'implémentations concrètes	une classe instancie directement une classe concrète dont elle dépend

5. Patrons de conception (design patterns) — les incontournables

Patron	Catégorie	Problème résolu
Singleton	création	garantir une unique instance globale d'une classe
Factory Method	création	déléguer la création d'objets à des sous-classes
Strategy	comportement	permuter un algorithme à l'exécution sans changer le code appelant
Observer	comportement	notifier automatiquement des objets abonnés d'un changement d'état
Decorator	structure	ajouter dynamiquement des responsabilités à un objet
Adapter	structure	rendre compatibles deux interfaces incompatibles
MVC	architecture	séparer données (Modèle), affichage (Vue) et logique de contrôle (Contrôleur)

Méthode d'exercice : repérer le problème central de l'énoncé (variation d'algorithme ? notification ? création complexe ?) puis faire correspondre au patron dont le « problème résolu » colle le mieux — ne jamais partir du nom du patron pour inventer le problème.

6. Tests logiciels

- **Test unitaire** : vérifie une unité isolée (fonction/méthode), avec des dépendances simulées (mocks/stubs).
- **Test d'intégration** : vérifie l'interaction correcte entre plusieurs modules.
- **Test système / de recette** : vérifie le système complet par rapport aux exigences.

- **TDD** : cycle « red → green → refactor » : écrire un test qui échoue, écrire le code minimal pour le faire passer, puis nettoyer le code.
- **Couverture** : couverture d'instructions (chaque ligne exécutée au moins une fois) vs couverture de branches (chaque condition vraie/fausse testée) — la couverture de branches est plus exigeante.

7. Estimation de charge

- **Points de fonction** : on compte les entrées, sorties, requêtes, fichiers internes/externes, pondérés par leur complexité, pour obtenir une taille fonctionnelle indépendante du langage.
- **COCOMO** : effort (en mois-homme) = $a \times (KLOC)^b$, où a et b dépendent du type de projet (organique, semi-détaché, embarqué) — savoir reconnaître la formule plutôt que la retenir par cœur si l'énoncé la fournit.

8. Agile / Scrum — vocabulaire à maîtriser

- **Product backlog** : liste priorisée de toutes les fonctionnalités souhaitées.
- **Sprint** : itération courte (2-4 semaines) qui produit un incrément livrable.
- **Sprint backlog** : sous-ensemble du backlog choisi pour le sprint courant.
- **Rôles** : Product Owner (priorise, représente le client), Scrum Master (facilite, lève les obstacles), équipe de développement.
- **Cérémonies** : sprint planning, daily stand-up, sprint review (démonstration), rétrospective.

9. Refactoring (quelques exemples classiques)

- **Extraire une méthode** : isoler un bloc de code répété dans sa propre méthode nommée.
- **Renommer** : améliorer la lisibilité sans changer le comportement.
- **Remplacer un conditionnel par du polymorphisme** : quand un switch/if teste le type d'un objet, déplacer chaque branche dans une sous-classe (souvent lié au principe Open/Closed).

10. Pièges classiques en examen

- Ne pas confondre agrégation et composition : la question à se poser est « la partie survit-elle à la destruction du tout ? ».
- Sur les diagrammes de séquence, ne pas oublier l'objet destinataire du message de retour (dashed arrow) quand demandé.
- Un design pattern n'est pas une solution magique : toujours justifier le choix par le problème de conception rencontré.
- Confondre exigence fonctionnelle et non-fonctionnelle est une erreur fréquente ; se demander « est-ce une action du système ou une qualité du système ? ».